МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Кафедра инфокоммуникаций

Основы кроссплатформенного программирования

Отчет по лабораторной работе №1.2

Исследование
возможностей Git для работы с
локальными репозиториями

Выполнил студент группы

ИВТ-б-о-21-1

Горшков В.И. « »_____20__г.

Подпись студента_____

Работа защищена « »_____20__г.

Проверил доцент
Кафедры инфокоммуникаций, старший преподаватель
Воронкин Р.А.

_____
(подпись)

Ставрополь 2022

Тема: Исследование возможностей Git для работы с локальными репозиториями.

Цель: исследовать базовые возможности системы контроля версий Git для работы с локальными репозиториями.

1) Необходимо создать новый репозиторий, в файле README.md указать информацию о обучающемся и скопировать репозиторий себе на компьютер.

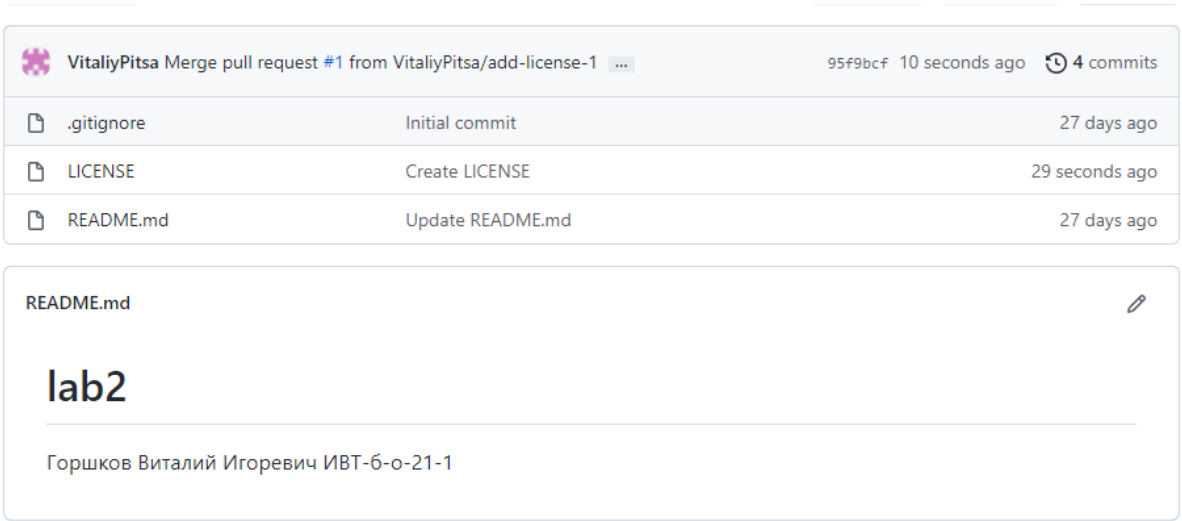

Рисунок 1 – Новый репозиторий

2) Проработка примеров в лабораторной работе.

1. Использование команды git log:

```
C:\lab2\lab2>git log
commit 95f9bcf8b4ddb87a7bafe29cc4580fdf6526ebf5 (HEAD -> main, origin/main, origin/HEAD)
Merge: 624dda2 c930af9
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Thu Apr 7 17:42:43 2022 +0300

    Merge pull request #1 from VitaliyPitsa/add-license-1

    Create LICENSE

commit c930af9d41a92e74649c6f740bd5fcbbfd034470 (origin/add-license-1)
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Thu Apr 7 17:42:24 2022 +0300

    Create LICENSE

commit 624dda29e86591a144adbbcdb1406c557a6c8cca
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Fri Mar 11 11:48:33 2022 +0300

    Update README.md

commit e8060cdfdf78e414588611b3404acfb7dded39d5
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Fri Mar 11 11:47:51 2022 +0300

    Initial commit
```

Рисунок 2 – Вывод команды git log в консоли

2.Использование команды git log -p -2(где 2 количество выведенных записей):

```
C:\lab2\lab2>git log -p -2
commit 95f9bcf8b4ddb87a7bafe29cc4580fdf6526ebf5 (HEAD -> main, origin/main, origin/HEAD)
Merge: 624dda2 c930af9
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Thu Apr 7 17:42:43 2022 +0300

    Merge pull request #1 from VitaliyPitsa/add-license-1

    Create LICENSE

commit c930af9d41a92e74649c6f740bd5fcbbfd034470 (origin/add-license-1)
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Thu Apr 7 17:42:24 2022 +0300

    Create LICENSE

diff --git a/LICENSE b/LICENSE
new file mode 100644
index 0000000..bcba127
--- /dev/null
+++ b/LICENSE
@@ -0,0 +1,21 @@
+MIT License
+
+Copyright (c) 2022 VitaliyPitsa
+
+Permission is hereby granted, free of charge, to any person obtaining a copy
+of this software and associated documentation files (the "Software"), to deal
+in the Software without restriction, including without limitation the rights
+to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
:
```

Рисунок 3– Вывод команды git log -p -2 в консоль

3.Использование команды git log –stat

```
C:\lab2\lab2>git log --stat
commit 95f9bcf8b4ddb87a7bafe29cc4580fdf6526ebf5 (HEAD -> main, origin/main, origin/HEAD)
Merge: 624dda2 c930af9
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Thu Apr 7 17:42:43 2022 +0300

    Merge pull request #1 from VitaliyPitsa/add-license-1

    Create LICENSE

commit c930af9d41a92e74649c6f740bd5fcbbfd034470 (origin/add-license-1)
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Thu Apr 7 17:42:24 2022 +0300

    Create LICENSE

 LICENSE | 21 +++++++++++++++++++++
 1 file changed, 21 insertions(+)

commit 624dda29e86591a144adbbcdb1406c557a6c8cca
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Fri Mar 11 11:48:33 2022 +0300

    Update README.md

 README.md | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)

commit e8060cdfdf78e414588611b3404acfb7dded39d5
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Fri Mar 11 11:47:51 2022 +0300

    Initial commit
```

Рисунок 4 – Вывод команды git log –stat

4. Использование команды git log --pretty=format:"%h - %an, %ar : %s"

```
C:\lab2\lab2>git log --pretty=format:"%h - %an, %ar : %s"
95f9bcf - VitaliyPitsa, 18 minutes ago : Merge pull request #1 from VitaliyPitsa/add-license-1
c930af9 - VitaliyPitsa, 18 minutes ago : Create LICENSE
624dda2 - VitaliyPitsa, 4 weeks ago : Update README.md
e8060cd - VitaliyPitsa, 4 weeks ago : Initial commit
```

Рисунок 5 – Вывод команды git log --pretty в консоль.

5.Использование команды git log –s

```
C:\lab2\lab2>git log -s
commit 95f9bcf8b4ddb87a7bafe29cc4580fdf6526ebf5 (HEAD -> main, origin/main, origin/HEAD)
Merge: 624dda2 c930af9
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Thu Apr 7 17:42:43 2022 +0300

    Merge pull request #1 from VitaliyPitsa/add-license-1

    Create LICENSE

commit c930af9d41a92e74649c6f740bd5fcbbfd034470 (origin/add-license-1)
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Thu Apr 7 17:42:24 2022 +0300

    Create LICENSE

commit 624dda29e86591a144adbbcdb1406c557a6c8cca
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Fri Mar 11 11:48:33 2022 +0300

    Update README.md

commit e8060cdfdf78e414588611b3404acfb7dded39d5
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Fri Mar 11 11:47:51 2022 +0300

    Initial commit

C:\lab2\lab2>_
```

Рисунок 6 – Вывод команды git log –s в консоль

6.Использование команды git remote –v

```
C:\lab2\lab2>git remote -v
origin  https://github.com/VitaliyPitsa/lab2.git (fetch)
origin  https://github.com/VitaliyPitsa/lab2.git (push)
```

Рисунок 7 – Вывод команды git remote –v в консоль

7. Использование команды git remote show origin

```
C:\lab2\lab2>git remote show origin
* remote origin
  Fetch URL: https://github.com/VitaliyPitsa/lab2.git
  Push  URL: https://github.com/VitaliyPitsa/lab2.git
  HEAD branch: main
  Remote branches:
    add-license-1 tracked
    main          tracked
  Local branch configured for 'git pull':
    main merges with remote main
  Local ref configured for 'git push':
    main pushes to main (up to date)
```

Рисунок 8 – Вывод команды git remote show origin

2) Написать небольшую программу на выбранном языке программирования(C++). Фиксировать изменения при написании программы

в локальном репозитории. Должно быть сделано не менее 7 коммитов, отмеченных не менее 3 тэгами.

```
Source.cpp
Лабораторная работа        (Глобальная область)        main()
1   #include<iostream>
2   #include<locale>
3   #include<time.h>
4   using namespace std;
5
6
7   int main()
8   {
9       setlocale(LC_ALL, "Russian");
10      cout << "hello word!\n";
11
12      return 0;
13  }
```

Рисунок 9 – Написанная программа

```
C:\lab2\lab2>git tag -a v1.1 -m "v.1.1"

C:\lab2\lab2>git tag
v1.1
```

Рисунок 10 – добавление тэгов

```
C:\lab2\lab2>git show
commit 737f4d84df3f310c4dd9c876b0a07dba3bc7afc7 (HEAD -> main, tag: v1.1, origin/main, origin/HEAD)
Author: VitaliyPitsa <vitalik.gorshkov.1913@mail.ru>
Date:   Thu Apr 7 18:40:10 2022 +0300

    test1

diff --git a/test1/.vs/ConsoleApplication1/v16/.suo b/test1/.vs/ConsoleApplication1/v16/.suo
index 843bf43..a95b197 100644
Binary files a/test1/.vs/ConsoleApplication1/v16/.suo and b/test1/.vs/ConsoleApplication1/v16/.suo differ
diff --git a/test1/.vs/ConsoleApplication1/v16/Browse.VC.db b/test1/.vs/ConsoleApplication1/v16/Browse.VC.db
index 5947ebd..cefe0bd 100644
Binary files a/test1/.vs/ConsoleApplication1/v16/Browse.VC.db and b/test1/.vs/ConsoleApplication1/v16/Browse.VC.db diffe
r
diff --git a/test1/ConsoleApplication1/ConsoleApplication1.cpp b/test1/ConsoleApplication1/ConsoleApplication1.cpp
index 044e070..4a6f34c 100644
--- a/test1/ConsoleApplication1/ConsoleApplication1.cpp
+++ b/test1/ConsoleApplication1/ConsoleApplication1.cpp
@@ -5,6 +5,15 @@
 int main()
 {
     std::cout << "Hello World!\n";
+    std::cout << "Hello World!\n";
+    std::cout << "Hello World!\n";
+    std::cout << "Hello World!\n";
+    std::cout << "Hello World!\n";
+    std::cout << "Hello World!\n";
+    std::cout << "Hello World!\n";
```

Рисунок 10 – просмотр изменений с помощью консоли

Просмотреть историю (журнал) хранилища командой git log. Например, с помощью команды git log --graph --pretty=oneline --abbrev-commit. Добавить скриншот консоли с выводом в отчет по лабораторной работе.

```
Microsoft Windows [Version 10.0.19043.1586]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Виталий>cd /d C:\lab2\lab2

C:\lab2\lab2>git log --graph --pretty=oneline --abbrev-commit
* 737f4d8 (HEAD -> main, tag: v1.1, origin/main, origin/HEAD) test1
* e8111f2 test1
*   95f9bcf Merge pull request #1 from VitaliyPitsa/add-license-1
|\
| * c930af9 (origin/add-license-1) Create LICENSE
|/
* 624dda2 Update README.md
* e8060cd Initial commit
```

Рисунок 11 – Вывод в консоль истории журнала

Просмотреть содержимое коммитов командой git show <ref>, где <ref>:

HEAD: последний коммит;

```
C:\lab2\lab2>git show head
commit 737f4d84df3f310c4dd9c876b0a07dba3bc7afc7 (HEAD -> main, tag: v1.1, origin/main, origin/HEAD)
Author: VitaliyPitsa <vitalik.gorshkov.1913@mail.ru>
Date:   Thu Apr 7 18:40:10 2022 +0300

    test1

diff --git a/test1/.vs/ConsoleApplication1/v16/.suo b/test1/.vs/ConsoleApplication1/v16/.suo
index 843bf43..a95b197 100644
Binary files a/test1/.vs/ConsoleApplication1/v16/.suo and b/test1/.vs/ConsoleApplication1/v16/.suo differ
diff --git a/test1/.vs/ConsoleApplication1/v16/Browse.VC.db b/test1/.vs/ConsoleApplication1/v16/Browse.VC.db
index 5947ebd..cefe0bd 100644
Binary files a/test1/.vs/ConsoleApplication1/v16/Browse.VC.db and b/test1/.vs/ConsoleApplication1/v16/Browse.VC.d
r
diff --git a/test1/ConsoleApplication1/ConsoleApplication1.cpp b/test1/ConsoleApplication1/ConsoleApplication1.cp
index 044e070..4a6f34c 100644
--- a/test1/ConsoleApplication1/ConsoleApplication1.cpp
+++ b/test1/ConsoleApplication1/ConsoleApplication1.cpp
@@ -5,6 +5,15 @@
 int main()
 {
     std::cout << "Hello World!\n";
+    std::cout << "Hello World!\n";
+    std::cout << "Hello World!\n";
+    std::cout << "Hello World!\n";
```

Рисунок 12-вывод последнего коммита

HEAD~1: предпоследний коммит (и т. д.);

```
:\lab2\lab2>git show head~1
ommit e8111f20126ada4fbd6e9048dcd873c18f486446
uthor: VitaliyPitsa <vitalik.gorshkov.1913@mail.ru>
ate:   Thu Apr 7 18:33:04 2022 +0300

    test1

iff --git a/test1/.vs/ConsoleApplication1/v16/.suo b/test1/.vs/ConsoleApplication1/v16/.suo
ew file mode 100644
ndex 0000000..843bf43
inary files /dev/null and b/test1/.vs/ConsoleApplication1/v16/.suo differ
iff --git a/test1/.vs/ConsoleApplication1/v16/Browse.VC.db b/test1/.vs/ConsoleApplication1/v16/Browse.VC.db
ew file mode 100644
ndex 0000000..5947ebd
inary files /dev/null and b/test1/.vs/ConsoleApplication1/v16/Browse.VC.db differ
iff --git a/test1/.vs/ConsoleApplication1/v16/ipch/AutoPCH/7389582f8fa964b8/CONSOLEAPPLICATION1.ipch b/test1
eApplication1/v16/ipch/AutoPCH/7389582f8fa964b8/CONSOLEAPPLICATION1.ipch
ew file mode 100644
ndex 0000000..d16178e
inary files /dev/null and b/test1/.vs/ConsoleApplication1/v16/ipch/AutoPCH/7389582f8fa964b8/CONSOLEAPPLICATI
ffer
iff --git a/test1/.vs/ConsoleApplication1/v16/ipch/AutoPCH/af7233bf2efac15e/CONSOLEAPPLICATION1.ipch b/test1
eApplication1/v16/ipch/AutoPCH/af7233bf2efac15e/CONSOLEAPPLICATION1.ipch
ew file mode 100644
ndex 0000000..5e3849f
inary files /dev/null and b/test1/.vs/ConsoleApplication1/v16/ipch/AutoPCH/af7233bf2efac15e/CONSOLEAPPLICATI
ffer
iff --git a/test1/ConsoleApplication1.sln b/test1/ConsoleApplication1.sln
```

Рисунок 13-предпоследний коммит

624dda2: коммит с указанным хэшем.

```
C:\lab2\lab2>git show 624dda2
commit 624dda29e86591a144adbbcdb1406c557a6c8cca
Author: VitaliyPitsa <99473215+VitaliyPitsa@users.noreply.github.com>
Date:   Fri Mar 11 11:48:33 2022 +0300

    Update README.md

diff --git a/README.md b/README.md
index d819d90..78e479c 100644
--- a/README.md
+++ b/README.md
@@ -1 +1,2 @@
-# lab2
\ No newline at end of file
+# lab2
+Горшков Виталий Игоревич ИВТ-б-о-21-1
```

Рисунок 14-вывод коммита по хэшу

Освоить возможность отката к данной версии.

Удалите весь код из одного из файлов программы репозитория, например main.cpp, и сохраните этот файл.

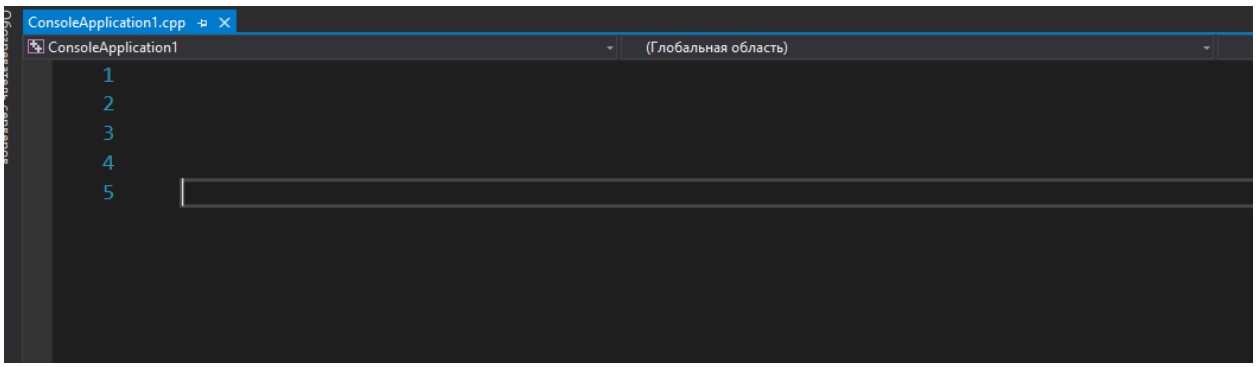

Рисунок 15-удалил весь код и сохранил изменения

Удалите все несохраненные изменения в файле командой: git checkout -- <имя_файла>, например git checkout -- main.cpp.

```
:\lab2\lab2>: git checkout -- ConsoleApplication1.sln
```

Рисунок 16 – удалил все несохраненные изменения в файле

Откатить состояние хранилища к предыдущей версии командой: git reset –hard HEAD~1

```
#include <iostream>

int main()
{
    std::cout << "Hello World!\n";
    std::cout << "Hello World!\n";
    std::cout << "Hello World!\n";
    std::cout << "Hello World!\n";
    std::cout << "Hello World!\n";
    std::cout << "Hello World!\n";
    std::cout << "Hello World!\n";
    std::cout << "Hello World!\n";
    std::cout << "Hello World!\n";
    std::cout << "Hello World!\n";
}
```

Рисунок 17-восстановленный код

Вопросы для защиты работы.

1. Как выполнить историю коммитов в Git? Какие существуют дополнительные опции для просмотра истории коммитов?

Наиболее простой и в то же время мощный инструмент для этого — команда git log. По умолчанию, без аргументов, git log выводит список коммитов созданных в данном репозитории в обратном хронологическом порядке. То есть самые последние коммиты показываются первыми.

Одна из опций, когда вы хотите увидеть сокращенную статистику для каждого коммита, вы можете использовать опцию –stat.

Вторая опция (одна из самых полезных аргументов) является -p или --patch, который показывает разницу (выводит патч), внесенную в каждый коммит. Так же вы можете ограничить количество записей ввыводе команды; используйте параметр -2 для вывода только двух записей (пример команды git log –p -2).

Третья действительно полезная опция это --pretty. Она меняет формат вывода. Существует несколько встроенных вариантов отображения. Опция oneline выводит каждый коммит в одну строку, что может быть очень удобным если вы просматриваете большое количество коммитов. К тому же, опции short, full и fuller делают вывод приблизительно в том же формате, но с меньшим или большим количеством информации соответственно.

Наиболее интересной опцией является format, которая позволяет указать формат для вывода информации. Особенно это может быть полезным, когда вы хотите сгенерировать вывод для автоматического анализа — так как вы указываете формат явно, он не будет изменен даже после обновления Git.

Для опции git log --pretty=format существуют различного рода опции для изменения формата отображения.

Опция	Описания вывода
%H	Хеш коммита
%h	Сокращенный хеш коммита
%T	Хеш дерева
%t	Сокращенный хеш дерева
%P	Хеш родителей
%p	Сокращенный хеш родителей
%an	Имя автора
%ae	Электронная почта автора
%ad	Дата автора (формат даты можно задать опцией --date=option)
%ar	Относительная дата автора
%cn	Имя коммитера
%ce	Электронная почта коммитера
%cd	Дата коммитера
%cr	Относительная дата коммитера
%s	Содержание

2. Как ограничить вывод при просмотре истории коммитов?

Для ограничения может использоваться функция git log <n>, где n число записей.

Также, существуют опции для ограничения вывода по времени, такие как --since и --until, они являются очень удобными. Например, следующая команда покажет список коммитов, сделанных за последние две недели:

git log --since=2.weeks

Это команда работает с большим количеством форматов — вы можете указать определенную дату вида 2008-01-15 или же относительную дату, например 2 years 1 day 3 minutes ago.

Также вы можете фильтровать список коммитов по заданным параметрам. Опция --author дает возможность фильтровать по автору коммита, а опция --grep (показывает только коммиты, сообщение которых содержит указанную строку) искать по ключевым словам в сообщении

коммита. Функция –S показывает только коммиты, в которых изменение в коде повлекло за собой добавление или удаление указанной строки.

3. Как внести изменения в уже сделанный коммит?

Внести изменения можно с помощью команды git commit --amend

Эта команда берёт индекс и применяет его к последнему коммиту. Если после последнего коммита не было никаких проиндексированных изменений (например, вы запустили приведённую команду сразу после предыдущего коммита), то состояние проекта будет абсолютно таким же и всё, что мы изменим, это комментарий к коммиту.

Для того, чтобы внести необходимые изменения - нам нужно проиндексировать их и выполнить комманду git commit --amend.

git commit -m 'initial commit'

git add forgotten_file

git commit --amend

Эффект от выполнения этой команды такой, как будто мы не выполнили предыдущий коммит, а еще раз выполнили команду git add и выполнили коммит.

4. Как отменить индексацию файла в Git?

Например, вы изменили два файла и хотите добавить их в разные коммиты, но случайно выполнили команду git add * и добавили в индекс оба. Как исключить из индекса один из них? Команда git status напомнит вам:

Прямо под текстом «Changes to be committed» говорится: используйте git reset HEAD <file> для исключения из индекса.

5. Как отменить изменения в файле?

С помощью команды git checkout -- <file>.

6. Что такое удаленный репозиторий Git?

Удалённый репозиторий это своего рода наше облако, в которое мы сохраняем те или иные изменения в нашей программе/коде/файлах.

7. Как выполнить просмотр удаленных репозиториев данного локального репозитория?

Для того, чтобы просмотреть список настроенных удалённых репозиториев, необходимо запустить команду git remote.

Также можно указать ключ -v, чтобы просмотреть адреса для чтения и записи, привязанные к репозиторию. Пример: git remote -v

8. Как добавить удаленный репозиторий для данного локального репозитория?

Для того, чтобы добавить удалённый репозиторий и присвоить ему имя (shortname), просто выполните команду git remote add <shortname> <url>.

9. Как выполнить отправку/получение изменений с удаленного репозитория?

Если необходимо получить изменения, которые есть у Пола, но нету у вас, вы можете выполнить команду git fetch <Название репозитория>. Важно отметить, что команда git fetch забирает данные в ваш локальный репозиторий, но не сливает их с какими-либо вашими наработками и не модифицирует то, над чем вы работаете в данный момент. Вам необходимо вручную слить эти данные с вашими, когда вы будете готовы.

Если ветка настроена на отслеживание удалённой ветки, то вы можете использовать команду git pull чтобы автоматически получить изменения из удалённой ветки и слить их со своей текущей. Выполнение

git pull, как правило, извлекает (fetch) данные с сервера, с которого вы изначально клонировали, и автоматически пытается слить (merge) их с кодом, над которым вы в данный момент работаете.

Чтобы отправить изменения на удалённый репозиторий необходимо отправить их в удалённый репозиторий. Команда для этого действия простая: git push <remote-name> <branch-name>.

10. Как выполнить просмотр удаленного репозитория?

Для просмотра удалённого репозитория, можно использовать команду git remote show <remote>.

11. Каково назначение тэгов Git?

Теги - это ссылки указывающие на определённые версии кода/написанной программы. Они удобно чтобы в случае чего вернутся к нужному моменту. Также при помощи тегов можно помечать важные моменты.

12. Как осуществляется работа с тэгами Git?

Просмотреть наличие тегов можно с помощью команды: git tag.

А назначить (указать, добавить тег) можно с помощью команды git tag -a v1.4(версия изначальная) -m "Название".

С помощью команды git show вы можете посмотреть данные тега вместе с коммитом: git show v1.4.

Отправка тегов, по умолчанию, команда git push не отправляет теги на удалённые сервера. После создания теги нужно отправлять явно на удалённый сервер. Процесс аналогичен отправке веток — достаточно выполнить команду git push origin <tagname>. Для отправки всех тегов можно использовать команду git push origin tags.

Для удаления тега в локальном репозитории достаточно выполнить команду git tag –d <tagname>. Например, удалить созданный ранее легковесный тег можно следующим образом: git tag -d v1.4-lw

Для удаления тега из внешнего репозитория используется команда git push origin --delete <tagname>.

Если вы хотите получить версии файлов, на которые указывает тег, то вы можете сделать git checkout для тега пример: git checkout -b version2 v2.0.0.

13. Самостоятельно изучите назначение флага --prune в командах git fetch и git push. Каково назначение этого флага?

Git fetch --prune команда получения всех изменений с репозитория GitHub.

В команде git push --prune удаляет удаленные ветки, у которых нет локального аналога.

Вывод: исследовал базовые возможности системы контроля версий git для работы с локальными репозиториями. Также, благодаря созданию тегов и пункту 7 лабораторной работы после изменения файлов освоил возможность отката к заданной версии.